

Improving GPU Scheduling Performance Based on Queue Adjustment and Fragmentation Curation

Abstract—As a representative example of hardware accelerators or coprocessors, Datacenter GPU is an essential hardware resource for AI research that provides significant computational power; however, it is also an expensive resource. Recently, the importance of hardware accelerators has significantly increased. The widespread adoption of transformer-based large language models (LLMs) has led to the use of dozens or even hundreds of NVIDIA Datacenter GPUs for training. Therefore, from an organizational perspective, the effective allocation and management of Datacenter GPU (hereafter referred to as GPU) resources is crucial. However, assigning training jobs to GPUs is equivalent to the well-known NP-hard job-scheduling problem. Therefore, most GPU schedulers adopt strategies that either assume specific conditions or aim to maximize or minimize certain metrics. In this study, we adjust the waiting queue and GPU resource pool situations to resolve cases in which queues are stuck, while separately addressing fragmentation issues that arise from prolonged operation. Hence, we introduce starvation-free and defragmentation algorithms. Even when readjusting the queue and GPU resource pool, the increased waiting time for some jobs in the queue or the overhead caused by resource pool reallocation should not exceed the absolute value of the overall gain in reducing the total waiting and execution times. Therefore, we adopted threshold-based monitoring using metric criteria. The starvation-free algorithm monitors the fair-share metric, whereas the defragmentation algorithm tracks the overhead of forcibly allocated jobs. We reduced the total wall time by more than 7.8% and achieved a 14.98% increase in the allocation rate for the most-allocated. Additionally, wall time reductions of 14%, 7.6%, and 8.9% were achieved for the compact, round robin, and MCTS schedulers, respectively.

Index Terms—Scheduling, GPU, fragmentation, job queue, scheduler, blockage, waiting queue

I. INTRODUCTION

Scheduling training or inference jobs on GPU servers is a crucial aspect of algorithm design. The correct term is data center GPU. However, as 'GPU' is widely used in this context, we will use 'GPU' throughout this paper to refer specifically to data center GPUs. GPU allocation rate and GPU utilization on the server are critical operational metrics affecting operational and research costs. Moreover, GPU resources are inherently limited to research and business groups prompting extensive studies for optimal job scheduling. However, there is no universal solution because most studies are based on specific conditions or highly idealized scenarios. This is because the problem arises from assigning tasks with unknown completion times to a queue that can handle tasks based on capacity, a well-known NP-hard combinatorial optimization problem. Most job-scheduling problems involve a combination of flow shop, job shop, single-machine, open shop, and K-machine

parallel scheduling [1]. Thus, most job-scheduling problems are handled by an approximation algorithm [2].

Nevertheless, research on schedulers for deep-learning model training and inference jobs continues to progress. Ye et al. [3] classified deep-learning workloads into several types based on workload characteristics and scheduling objectives. The first classification framework is based on workload characteristics and categorized workloads into training, inference, and other categories. The second classification is applied according to the specific characteristics of each workload. For training workloads, this classification is further divided into scheduling objectives and resource utilization managers. The scheduling objective is subdivided into efficiency, fairness, and deadline guarantees. Sia [4] is a state-of-the-art (SOTA) scheduler for deep-learning training workloads. This study demonstrated strong performance in terms of utilization, fairness, and job completion time (JCT). Another SOTA scheduler is Lucid [5], which relies on measured metrics by analyzing jobs using a profiler in a multistate environment. To the best of our knowledge, most GPU schedulers for deep-learning adopt similar approaches. GPU allocation scheduling is an NP-hard problem. Therefore, if job characteristics change, scheduling policies or schedulers should be adjusted accordingly.

In contrast, we aimed to improve the efficiency of GPU server clusters by optimizing the job wait queue and fragmented GPU allocation. When a job requests the maximum number of GPUs on a server, it may experience long wait times if the server has insufficient resources, resulting in starvation. This also delays subsequent jobs even when servers with adequate GPUs are available. Such situations frequently occur during prolonged deep-learning training, reducing the overall allocation rate and causing dissatisfaction among the model developers.

In this study, we propose a novel approach that directly adjusts job queues and the GPU server pool state to enhance resource allocation and minimize the fairness compromise. To ensure scheduling fairness, we introduced a mechanism for dynamically adjusting the job queue priorities at specific intervals. Additionally, we optimized the GPU server pool status to schedule starved jobs without modifying the job queue. These combined methods form our queue adjustment and fragmentation-curation framework. Our algorithm achieved a 7.8% reduction in JCT and improved the allocation rate by 14.98 percentage points. We validated our approach using augmented job logs generated from real GPU server pool data.

Compared to other schedulers, the contributions of our study are as follows:

- We propose a novel mechanism for adjusting job wait queues to improve GPU resource scheduling efficiency while ensuring fair task prioritization across workloads.
- Advanced queue and fragmentation curation consistently deliver high performance, regardless of the job characteristics or scheduler type, making it adaptable to diverse job request patterns.
- Our approach determines the optimal coefficients based on job and scheduler characteristics. To validate its effectiveness, we developed a scalable simulator for rapid evaluation.

The remainder of this paper is organized as follows: Section II presents several related studies. The following section explains why job-waiting queue adjustments are required. Section IV shows details of queue adjustment and fragmentation curation in a mathematical manner. In the subsequent section, we demonstrate how effective queue adjustment and fragmentation curation maintain a high efficiency within the GPU server pool, even with varying job characteristics. In addition, we present the performance results of applying our mechanism to job logs that have been enhanced using actual data. Finally, we summarize this study and discuss potential directions for future development in Section VII.

II. RELATED WORKS

A. Lucid

Lucid [5] introduced an efficient and user-friendly approach for deep-learning job scheduling based on three foundational principles: ease of integration, scalability, and transparency. It avoids intrusive modifications by operating without preemption and eliminates the need for user intervention or adjustments to existing deep-learning frameworks. These design choices effectively address challenges, such as operational rigidity, error susceptibility, and high integration or maintenance overheads. Additionally, Lucid was designed to handle complex and large-scale workloads efficiently while providing clear and adjustable operations for system administrators. Lucid’s primary focus was to reduce the average JCT for training workloads, catering to the demands of deep-learning users. In addition to improving resource efficiency, it also offers prompt debugging feedback. Looking ahead, Lucid aims to support additional scheduling objectives, such as ensuring fairness and meeting service-level agreements, thereby expanding its applicability to diverse workload scenarios.

B. Sia

Sia [4] is a scheduler specifically designed to handle resource-adaptive deep-learning jobs in heterogeneous GPU clusters. It addresses the challenges of balancing GPU type, count, and batch size assignments dynamically, which is particularly difficult because of the vast search space and diverse scaling behavior of different jobs. By leveraging an integer linear programming (ILP) formulation and pragmatically reducing the search space, Sia efficiently assigns resources while minimizing overhead. In addition, Sia employs an innovative throughput modeling approach that relies on minimal initial

profiling and dynamically refines its predictions, enabling effective resource allocation without significant profiling costs. Furthermore, Sia proved effective in maintaining fairness metrics and efficiently scaling complex hybrid parallel jobs, such as Megatron-style models [6], while maintaining robust performance across varying cluster sizes and workloads. These results highlight Sia’s scalability, adaptability, and superior performance in addressing the complex challenges associated with heterogeneous resources and job variability.

C. Algorithms for Preemptive Co-scheduling of Kernels on GPUs

Algorithms for the preemptive co-scheduling of kernels on GPUs [7] aim to minimize the number of preemptions while achieving an optimal makespan. This study demonstrates that the optimal preemptive makespan can be computed efficiently using a linear programming solution. However, determining the minimum number of preemptions among all the solutions with an optimal makespan has been proven to be NP-hard. These findings highlight the effectiveness of graph-based scheduling techniques in balancing computational efficiency with minimal disruption in high-demand GPU environments.

III. MOTIVATION

Interestingly, technology with significantly advanced deep learning has also led to inefficiencies in GPU server pools. Container [8]-based, reproducible, and shareable training environments—enabled by services such as Docker Hub [9] and Hugging Face [10]—have greatly accelerated deep-learning research around the globe. With various ML frameworks, libraries, and fundamental neural network architectures for deep learning being widely shared, unnecessary time spent on reproducibility has been minimized. Consequently, almost all researchers can focus on refining their algorithms in a consistent environment. The GPU-dependent components of model architecture are often abstracted or hidden.

Because GPU access is abstracted, containers typically interact with GPUs at the card level. Consequently, despite hardware support for time slicing to improve the GPU allocation rate, it cannot be flexibly utilized during model training or GPU server pool management. Moreover, as the scale of deep-learning models grows, the need to use multiple GPUs simultaneously has become commonplace. When managing a GPU server pool, an endless stream of jobs is continuously submitted, each with varying completion times and GPU counts. Consequently, fragmentation occurs within the GPU server pool, leading to inefficiency. Fragmentation occurs in the following situations:

Figs. 1 and 2 show the status of the GPU server pool and waiting job queue, respectively. A scheduler that uses the most-allocated manager can allocate tasks t_0 and t_1 . However, it cannot schedule t_2 because t_2 depends on completing other jobs to free up GPU slots. This implies that tasks t_3 , t_4 , and t_5 can only be scheduled afterward (Fig. 3).

As illustrated in Figs. 3, despite having sufficient GPU slots to schedule t_3 , t_4 , and t_5 , these jobs cannot be scheduled

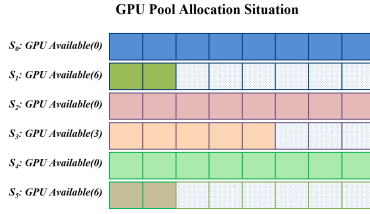


Fig. 1. GPU server pool status before newly submitted jobs are scheduled.

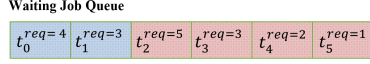


Fig. 2. Waiting job queue status before newly submitted jobs are scheduled.

due to t_2 , which is blockage. This situation occurs because previously allocated jobs cause the GPU server pool to become fragmented. Fragmentation in the GPU server pool occurs continuously during the operation and is occasionally resolved. The worst case is occurs when there are seven GPUs available on all GPU servers, and the blockage requests eight GPUs. In this case, the GPU server pool hits the lowest GPU allocation rate before any job is completed. In this study, we experimentally investigated this phenomenon. We allocated jobs based on the most-allocated scheduling approach using job logs recorded from an actual GPU server pool. The logs were collected over approximately three months. We analyzed the scheduling conditions according to the required GPU count. The experiment assumed a GPU server pool composed of Nvidia A30 and A100 GPUs with a single job-waiting queue and no job cancellations while waiting.

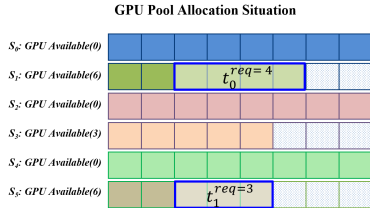


Fig. 3. GPU server pool status after some jobs are scheduled.

The characteristics of the job log are shown in Fig. 4, which illustrates the requested number of GPUs and the training time for each job. Because of the mixture of A30 and A100 GPUs, there is an exceptionally high number of jobs requesting four GPUs, whereas the other GPU request counts follow a uniform distribution. The training time follows a Weibull minimum distribution.

We examined the frequency of blockages using scheduling analysis. In particular, we observed the overall allocation rate of the GPU server pool when job groups with different GPU request counts were scheduled. Blockages have the greatest impact on jobs with low GPU request counts. For jobs requesting only one or two GPUs, scheduling can occur even at a high GPU server pool allocation rate. However, if a

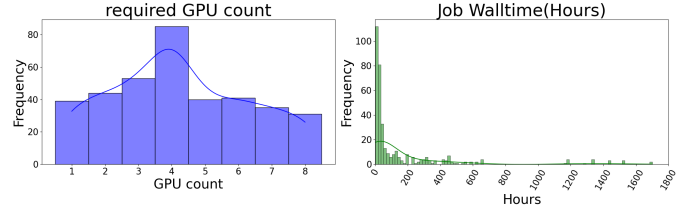


Fig. 4. Actual job log characteristics.

blockage is in front of the waiting queue, these jobs cannot be allocated until the blockage is resolved, which often results in allocation when the overall allocation rate of the server pool decreases to a certain level. Fig.5 shows that jobs requesting between one and eight GPUs are allocated at similar pool allocation rates, suggesting a high frequency of blockage.

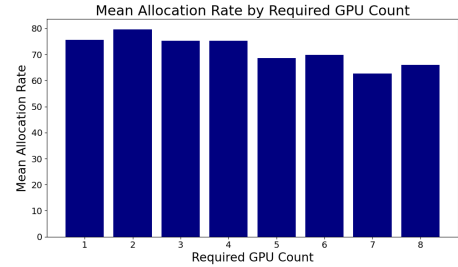


Fig. 5. GPU server pool allocation rate when job has been allocated

The challenge of optimally scheduling an infinite number of jobs with unknown completion and scheduling request times is recognized as NP-hard. Consequently, this problem cannot be entirely resolved using scheduling algorithms alone. As a result, this study prioritizes addressing blockages instead of enhancing the scheduling algorithms. Consequently, we introduce new mechanisms called queue adjustment and fragmentation curation.

We developed an in-house simulation for job scheduling and server adjustment to facilitate the analysis and rapid repeated development of mechanisms to resolve blockages. It parses job logs and responds to scheduling using various methods. New scheduling methodologies can be added without affecting existing mechanisms. This implies that the queue and fragmentation adjustments operate independently of the job-scheduling method.

IV. RESOLVING BLOCKAGE IN JOB QUEUE AND GPU FRAGMENTATION

We propose two approaches to address blockages in the job-waiting queue and fragmentation of the GPU server pool. First, we introduce a starvation-free algorithm to reduce blockages. Second, we develop a preemptive task reallocation algorithm to minimize fragmentation in the GPU server pool. Both algorithms aim to optimize the usage of the GPU server pool. Each algorithm functions independently and has its own set of trade-offs. These trade-offs are determined using the coefficients applied to each algorithm.

A. Starvation-free Algorithm

Different job schedulers have unique goals and characteristics that are designed for specific scheduling strategies. The advanced queue and fragmentation management proposed in this study aims to ensure a high allocation rate for the GPU server pool, regardless of the type or method of the scheduler used. The starvation-free algorithm and preemptive task reallocation algorithm function before scheduler execution. Together, these two algorithms optimize the job-waiting queue and server states, enabling an optimal scheduling performance independent of the scheduler type or mechanism.

Blockages frequently occur during GPU server pool operations and are eventually resolved; however, the resolution can take time. This delay negatively affects the overall allocation rate. Scheduling to prevent blockages is an NP-hard problem, and it is nearly impossible to predict when blockages will occur. This is due to the unpredictable nature of the types of jobs requesting GPUs and the resources required in the real world. So, scheduling to prevent blockages is an indirect approach. The starvation-free algorithm addresses blockage issues by directly modifying the priorities of jobs in the waiting queue. We also consider the trade-offs that arise with these modifications using specific coefficients. Although adjusting the queue can enable jobs blocked by a blockage to be scheduled, there is a trade-off that, in the worst-case scenario, jobs associated with the blockage may never be allocated. This algorithm was designed to prevent starvation in top-priority jobs. Since, arbitrarily removing blockages or adjusting priorities can lead to fairness issues.

Let P^* represent the adjusted priority value of tasks $t_0^*, t_1^*, t_2^*, \dots, t_n^*$. P^* is calculated and reassigned before the scheduling is initiated. The scheduler utilizes modified job priorities. The adjusted job priority P^* is defined as

$$P^* = \begin{cases} P + \alpha A \times R, & \text{if } \beta > AR \\ P, & \text{otherwise} \end{cases} \quad (1)$$

The first term, P represents the original priority of tasks $t_0, t_1, \dots, t_n$, where n is three times the number of servers. Specifically, the priorities are defined as follows: $p_0 = 1$, $p_1 = 0.5$, and $p_2 = 0.25$, with each subsequent priority decreasing by half; a higher value indicates greater priority than a lower one. The second term consists of the variables $\alpha A \times R$ which represent the starvation index. Here, α is the coefficient that dictates how quickly the starvation index gains importance. For example, to ensure that the priority of the second job in the waiting queue increases within six hours, assuming that the R value is set to one, the α value should be set to at least 0.013889. In numerical terms, when P_1 is 0.5, the calculation $0.013889 \times 36 \times 1 = 0.500004$ results in an updated priority of 1.000004, surpassing the P_0 value of 1, and thereby shifting the priority. In this context, A refers to the age of the task, which increases by one every ten minutes, and R is the resource suitability index. A is reset when a job is scheduled. The value of R is one when the number of required accelerators exactly matches the available number of

accelerators. However, it decreases by 0.1 for each additional accelerator. Finally, β is the minimum allocation rate that determines whether the starvation-free algorithm should be activated. The starvation-free algorithm is triggered if the allocation rate AR falls below this threshold. Algorithm 1 shows the implementation of a starvation-free algorithm using (1).

Algorithm 1 Starvation-free Algorithm

Input:

Server count : n , Wait queue length : l_{wq}
Servers : $S_i \in \{S_0, S_1, \dots, S_{n-1}\}$,
Accelerator count in S_i : $M_i \in \{M_0, M_1, \dots, M_{n-1}\}$,
Resource suitability index : $R_i = 0, \forall i \in \{1, 2, \dots, 8\}$,
Wait Queue : $WQ_i \in \{WQ_0, WQ_1, \dots, WQ_{l-1}\}$,
Task in WQ : $t_i \in \{t_0, t_1, \dots, t_{l-1}\}$,
Accelerator request of t_i : a_{t_i}
Wait Age : $A_i \in \{A_0, A_1, \dots, A_{m-1}\}$

Procedure: Updated wait queues

```

1: if latest allocation  $\geq \beta$  then
2:   return
3: end if
4:  $R \leftarrow 0$ 
5: for  $S_{i \leftarrow 0}$  to  $S_{n-1}$  do
6:   for  $j \leftarrow 1$  to  $M_i$  do
7:     if  $a_{S_i} > j$  then
8:        $R_j \leftarrow \max\{R_j, 1 - (a_{S_i} - j) \times 0.1\}$ 
9:     end if
10:  end for
11: end for
12: for  $i \leftarrow 0$  to  $l_{wq} - 1$  do
13:    $P_i \leftarrow 1/2^i$ 
14:    $P_i^* \leftarrow P_i + \alpha A_i \times R_{a_{t_i}}$ 
15: end for
16:  $P_i^*, i = \max(P^*)$ 
17:  $P_0 \leftarrow P_i^*$ 
18: for  $j \leftarrow 1$  to  $i$  do
19:    $P_j \leftarrow P_{j-1}^*$ 
20: end for

```

B. Preemptive task reallocation Algorithm

The preemptive nature of accelerator-based learning tasks means that they can be paused and resumed from the paused points using checkpoints. This characteristic can be utilized to resolve the fragmentation in the server pool. Resolving fragmentation is not about adjusting to the current queue but rather about consolidating preemptive tasks in one place to increase the number of servers without job allocations. Preemptive jobs are moved to maximize the number of servers with no assigned jobs. Dynamic programming is used to determine the optimal reallocation. There are two main trade-offs when using dynamic programming for job reallocation. The first is the execution time of the dynamic programming algorithm. The goal is not to find the optimal placement for all

preemptive jobs, but rather to maximize the number of servers with no assigned jobs. Given the small scale of the server pool, redundant optimal placements can be found quickly; however, the total number of possible configurations for all preemptive jobs is extremely large. Consequently, if all configurations are considered, the time cost of dynamic programming may outweigh the benefits of defragmentation. The second trade-off is the time required for reallocation. Using checkpoints for reallocation does not operate at the OS kernel level; instead, it involves stopping the job, performing reallocation, and resuming execution, because there is downtime for this operation. It is essential to find an appropriate balance between total downtime and the benefits gained from reallocation.

$$dp(i, a_0, a_1, \dots, a_{n-1}, \text{calls}) = F_i \quad (2)$$

Equation (2) represents the dynamic programming, dp stands for dynamic programming, for the maximum number of servers F_i with exactly the total accelerator count in the server left after the reallocation task i , where $a_0, a_1, \dots, a_{n-1}$ are the available hardware accelerators on each server, and calls denotes the number of recursive calls. Each server S_j begins with tasks already allocated, implying that its available hardware accelerators are reduced based on the tasks already placed on that server. Equation (3) shows an initial status of dp .

$$dp(0, a_0, a_1, \dots, a_{n-1}, 0) = F_0 \quad (3)$$

where a_j is the current number of available hardware accelerators on the server j . The maximum capacity of each server is $M_j = 4$ or $M_j = 8$ depending on the server type. The dp process begins only if the waiting queue length is satisfied; that is, the wait queue (WQ) length $\geq \omega$. t_r is a running task on the server that performs preemptive operation. r denotes recursive call depth.

$$\begin{aligned} dp(i, a'_0, a'_1, \dots, a'_{n-1}, \text{calls}) = \\ \max_{i \neq S_i} (dp(i-1, a_0, a_1, \dots, a_{n-1}, \text{calls} - 1), \\ dp(i, a_0, a_1, \dots, a_{n-1}, \text{calls}) + \sum_{j=0}^{n-1} f(a_j, t_r, M_j)) \quad (4) \\ , \text{ where } WQ \geq \omega. \end{aligned}$$

where $i \neq S_i$ means that task t_i does not need to be reassigned to its original server S_i . Once the calls reaches δ , the recursion will stop. When task t_i is reassigned from server S_i , the resources used by the task are released; that is, $a'_{S_i} = a_{S_i} + a_{t_i}$. After reassigning task t_i to server j (with $j \neq S_i$), the available hardware accelerators on server j are updated as $a'_j = a_j - a_{t_i}$. In addition, $a'_j \geq 0$ must hold, which implies that the server cannot be overcommitted.

$$f(a_j, t_r, M_j) = \begin{cases} 1 & \text{if } a'_j = M_j, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

Equation (5) checks if, after the task reassignment, server j has exactly the maximum hardware accelerators left (either four or eight, depending on M_j).

$$dp(m, a_0, a_1, \dots, a_{n-1}, \text{calls}) = F_m, \text{ if } F_m > F_0. \quad (6)$$

Equation (6) presents the final state that reallocates preemptive tasks to the most reasonable server as a result of dynamic programming. Therefore, Algorithm 2 represents the detailed sequence of dynamic programming.

Algorithm 2 Preemptive task reallocation Algorithm

Input:

Server count : n , Wait queue length : l_{wq} ,
 Memorization cache : mc , Full empty server count : F
 Static call count : $\text{calls} \leftarrow 0$, Recursive call depth : r ,
 Servers : $S_i \in \{S_0, S_1, \dots, S_{n-1}\}$,
 Shadow Servers : $S'_i \in \{S'_0, S'_1, \dots, S'_{n-1}\} \leftarrow S$,
 Accelerator count in S_i : $M_i \in \{M_0, M_1, \dots, M_{n-1}\}$,
 Available accelerator count in S'_i : a_i ,
 Preemptive task : $t_j \in \{t_0, t_1, \dots, t_{j-1}\}$
 Wait Queue : $wq_i \in \{wq_0, wq_1, \dots, wq_{l-1}\}$,

Output:

Max Full empty Server count : F , Final status servers S'

Procedure(DP for reallocation): $dp(r, F, \text{calls})$

```

1: if  $l_{wq} < \omega$  then
2:   return -1
3: end if
4:
5: if  $\text{calls} > \delta$  then
6:   return  $F$ 
7: end if
8:  $\text{calls} \leftarrow \text{calls} + 1$ 
9: if  $MC[\text{allocation\_state}]$  existed then
10:  return  $MC[\text{allocation\_state}]$ 
11: end if
12: if  $\text{calls} == 0$  then
13:   Clear  $MC$ 
14: end if
15: if  $r == l_{wq}$  then
16:   return  $F$ 
17: end if
18: for  $i \leftarrow 0$  to  $n - 1$  do
19:    $F \leftarrow \max(F, \sum_{j=0}^{n-1} f(a_j, t_r, M_j))$ 
20:    $MC[\text{allocation\_state}] \leftarrow F$ 
21:    $F \leftarrow dp(r + 1, F, \text{calls})$ 
22: end for
23:  $F \leftarrow dp(r + 1, F, \text{calls})$ 

```

The queue adjustment and fragmentation curation proposed in this study aim to maintain a high allocation rate for the GPU server pool, regardless of the scheduler type or method. The starvation-free and preemptive task reallocation algorithms function before scheduler execution. Together, these two algorithms optimize the job-waiting queue and server states,

enabling an optimal scheduling performance independent of the scheduler type or mechanism.

The two algorithms operate independently of the scheduler, and are executed before the scheduler, which operates at regular intervals. They rely solely on the job-waiting queue and current job allocation status within the server pool to determine their actions. Because the starvation-free and preemptive task reallocation algorithms are not interdependent, they can function separately, and users may opt to run only one of the algorithms if desired. This study directly manages the job-waiting queue and jobs allocated to the servers, making it incompatible with schedulers that also perform job scheduling for these elements. Therefore, this study did not target schedulers that rely on cumulative metrics or contexts.

V. OPTIMIZATION OF COEFFICIENT α , β , ω , AND δ

The starvation-free and preemptive task reallocation algorithms utilize the coefficients α , β , ω , and δ to control trade-offs. This section analyzes the impact of the four coefficients on performance improvement and derives an objective function to determine the optimal values for each coefficient. This approach enables the calculation of coefficients tailored to various job behaviors.

A. Correlation between performance and coefficients

All four coefficients must be calculated and applied to ensure an accurate operation. The impact of each coefficient was measured using job logs to determine the optimal values. The coefficient ranges are listed in Table. I.

TABLE I
COEFFICIENT RANGE

Coefficient	α	β	ω	δ
Range	0.01 to 1.99	80% to 95%	10 to 100	100k to 500k
Step	0.01	5%	10	100k

First, we analyze the impact of each coefficient on the completion time of all jobs in the log using SHAP [11] values. The results of the SHAP analysis are shown in Fig. 6. The four-coefficient SHAP value analysis yielded 40,086 possible combinations.

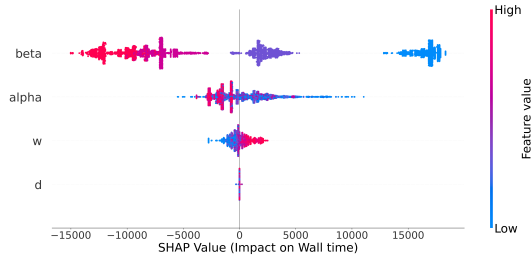


Fig. 6. SHAP value analysis of coefficients.

Fig. 6 analyzes the impact of each coefficient on overall job completion time, or wall time. The coefficient β has a strong negative correlation, where higher values of β correspond to

reduced wall time. The coefficient α also negatively correlates with the wall time, but exerts a weaker influence than β . The coefficient δ shows no significant positive or negative correlation with wall time. The coefficient ω exhibits a weak positive correlation, indicating that lower values of ω slightly reduce the wall time, although its influence is less than that of α .

B. Trade-off analysis and definition of related metrics

To optimize the four coefficients, it is essential to calculate the trade-offs. The coefficients α and β help eliminate blockages but may also cause blocked jobs to remain unscheduled indefinitely, leading to starvation or extended delays. To prevent this, the appropriate coefficients must be set. Thus, the cumulative waiting time for each job is grouped by the number of requested accelerators and calculated as a trade-off, which we define as the accumulated average distance (AAD).

$$AAD = \sum_{k=1}^8 \frac{(WT'_k - WT_k)}{2^{8-k}} \quad (7)$$

Equation (7) measures the weighted average difference between the waiting times before and after applying α and β . Let the original sum of the waiting times of the jobs grouped by the requested hardware accelerators count be denoted by $WT_1, \dots, WT_8$.

After applying α and β , the updated waiting times are $WT'_1, WT'_2, \dots, WT'_8$. The distance between these waiting times is calculated as the average accumulated distance. In addition, the distance is halved each time the number of requested accelerators decreases.

The accumulated maximum distance is the value selected from the waiting time differences. w_t represents the set of waiting times for jobs based on the requested hardware accelerator count. Therefore, w'_t represents the set of maximum waiting times for each requested hardware accelerator count.

Equation (8) shows an accumulated maximum distance (AMD).

$$AMD = \sum_{k=1}^8 \max \left(\frac{w'_t - w_t}{2^{8-k}} \right) \quad (8)$$

The computation of these metrics ensures that both the average and maximum deviations in task waiting times are considered, enforcing fairness in scheduling while preventing any significant performance deterioration owing to starvation or excessive waiting.

The preemptive task reallocation algorithm takes longer when dynamic programming is performed, indicating the additional time required compared to not using this algorithm. The trade-off of the preemptive task reallocation algorithm is measured using the algorithm execution time (AET).

C. Objective functions

The coefficients α and β are adjusted to minimize the wall time, AAD, and AMD. To determine the optimal values of these two coefficients, a grid search over a range of α

and β values is performed. The wall time, AAD, and AMD are computed for each combination of α and β . All three metrics are normalized for the min-max scaler to ensure fair comparison across metrics. An objective function is defined to aggregate the normalized values of the three. The objective function of the starvation-free algorithm is given by (9). The objective function is a simple formation. Hence, the minimum value is found using the min-max scalar. δ and ω are calculated using the min-max scalar.

$$Obj_i^{sfa} = walltime_i^{norm} + AAD_i^{norm} + AMD_i^{norm} \quad (9)$$

The objective function for δ and ω aims to minimize both the wall time and overhead introduced by preemptive task reallocation. This is measured indirectly using dynamic programming execution time (ms) (DPE). In addition, this algorithm aims to maximize preemption gain. The preemption gain is the measured gap between the original and algorithm-applied wall times. It marks as PMG. Equation (10) represents the objective function for δ and ω .

$$Obj_i^{ptr} = walltime_i^{norm} + DPE_i^{norm} + PMG_i^{norm} \quad (10)$$

VI. EXPERIMENTAL EVALUATION

Queue adjustment and fragmentation curation are simple, flexible, and powerful mechanism. To the best of our knowledge, no existing method directly addresses blockages in the job-waiting queue and fragmentation within the servers. To validate the flexibility and performance of the queue adjustment and fragmentation curation, we identified the optimized coefficients for various job distributions and evaluated the improvements in scheduling performance using these coefficients. To identify the optimal coefficients for various job distributions, we simulated the accelerator requests and job execution times using eight distributions: normal, exponential, log-normal, gamma, beta, Weibull minimum, uniform, and Poisson. All generated job logs contained 1,000 jobs. The ranges of the coefficients in the search space are listed in Table. I However, only the range for ω was modified to span from 20 to 200 in increments of 20.

To measure the performance improvement, we used two kinds of job logs: actual logs from Section III and augmented job logs based on the actual logs. These logs contained 368 and 3,001 job entries, respectively. The evaluation used two job logs to compare the results before and after applying queue adjustment and fragmentation curation. The four coefficients used were identical to those in the search space above. The characteristics of the augmented job log are presented in Fig. 7. It includes the requested number of GPUs and wall time for each job.

A simulator was employed to determine the optimal coefficients and compare the performance of the mechanism's. The simulations were executed on a Windows 11 system with an Intel Core Ultra 7 155H 3.80 GHz processor and 32GB of RAM, which was developed in C++ using Visual Studio Community 2022.

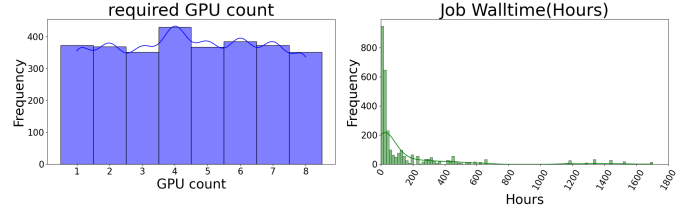


Fig. 7. Augmented job log characteristics.

A. Optimal value of coefficient α , β , ω , and δ

The simulation results for the eight types of distribution job logs represent the optimal value of each coefficient. Fig. 8. represents the characteristics of each job log. Table II. shows the optimal values of the coefficients α , β , ω , and δ for each distribution and the time reduction achieved by the algorithm. Additionally, Table II shows that the optimal coefficients vary according to the job log characteristics. This suggests that rather than using fixed coefficients, it is beneficial to adjust them according to the specific characteristics of the workload.

The results demonstrate that queue adjustment and fragmentation curation operate effectively across various types of workloads. Furthermore, it highlights the importance of fine-tuning the coefficients to match the specific characteristics of each workload.

TABLE II
OPTIMAL VALUE OF COEFFICIENTS

Distribution methodology	α	β	ω	δ
Normal	0.53	85%	60	200k
Exponential	0.44	90%	200	400k
Log-normal	1.63	95%	20	100k
Gamma	0.22	95%	80	400k
Beta	0.26	90%	180	100k
Weibull minimum	0.55	90%	120	400k
Uniform	0.5	95%	200	400k
Poisson	0.22	90%	100	300k

B. Evaluation of scheduling improvement

The previous optimal search method for coefficients allowed us to define the coefficients for both the actual and augmented job logs. The coefficients are listed in Table III, Figs. 9(A) and 9(B) illustrate the effects of the coefficients on wall time and the trade-off metric when simulating using the actual job log.

TABLE III
OPTIMAL COEFFICIENT VALUES FOR JOB LOG

Coefficient	Actual job log	Augmented job log
α	0.03	0.72
β	90 %	90%
ω	10	200
δ	200k	300k

Figs. 9(A) and 9(B) illustrate the effects of the optimal coefficients on the allocation rate and the utilization rate when simulating using both job logs. The X-axis represents the time,

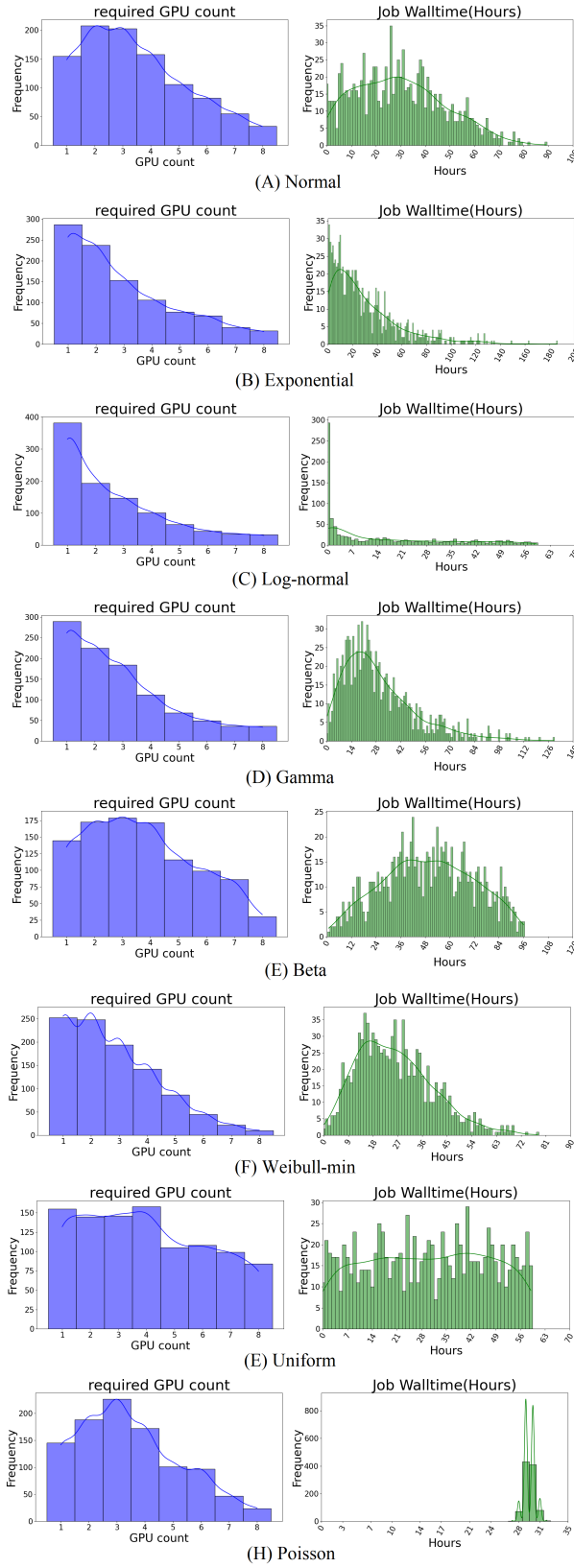


Fig. 8. Eight types of job distribution characteristics.

in which one unit corresponds to 10 min. GPU server pool allocation and total utilization were also obtained. In Figs. 9(C) and 9(D), both the allocation rate and utilization of the actual job log shift to the right, indicating a higher allocation rate and utilization.

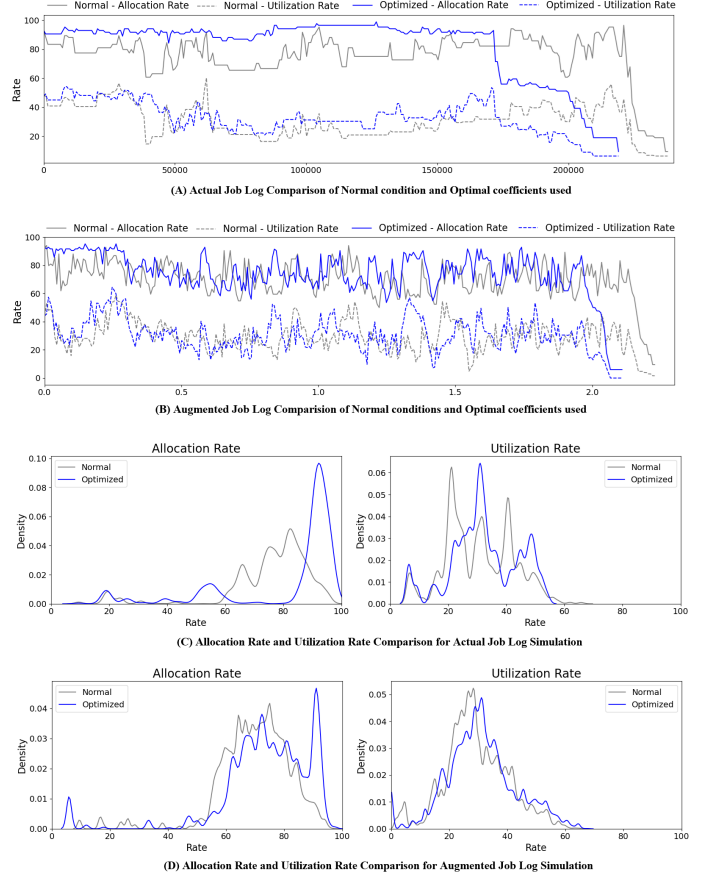


Fig. 9. Simulation results comparison for two job log.

Before applying the queue adjustment and fragmentation curation wall, the times of the actual job log and augmented job were 165 days, 2 hours, and 1 min, and 1,547 days, 6 hours, and 1 min, respectively. After applying the algorithm, the wall times of the two job logs were reduced by approximately 13 and 80 days, respectively. The actual job log achieved a 7.8% performance gain in wall time. A 4.9% performance gain was achieved for augmented job logs archived. The results are shown in Fig. 9(A), Fig. 9(B), and Table IV.

Table IV lists the wall times for each combination of algorithms. The simulation results for the actual and augmented job logs exhibited different behaviors. An augmented job log was generated based on the actual job log. The first part of the augmented job log consists of 368 jobs identical to those in the actual job log. Subsequently, augmentation was performed to reflect the behavior of the actual job logs. As a result of this augmentation, the dominance of the job group requiring four GPUs was diminished. Consequently, the augmented job log was approximately 8.1 times larger than the actual job log. A

large volume of logs indicates that fragmentation in the GPU server pool intensifies as the workload accumulates over time.

TABLE IV
WALLTIME OF SIMULATION RESULTS

Applied Algorithm	Actual job log	Augmented job log
Normal	165d 02h 01m	1,547d 06h 01m
All algorithms	152d 02h 20m	1,464d 08h 07m
Starvation-free algorithm only	151d 01h 38m	1,464d 06h 22m
Preemptive task reallocation algorithm only	165d 10h 01m	1,545d 17h 03m

The simulation results for the actual and augmented job logs exhibited different behaviors. An augmented job log was generated based on the actual job log. The first part of the augmented job log consists of 368 jobs identical to those in the actual job log. Subsequently, augmentation was performed to reflect the behavior of the actual job log. As a result of this augmentation, the dominance of the job group requiring four GPUs was diminished. Table IV shows the differences in the wall time when both algorithms are applied together compared to when each algorithm is applied individually. The reduction in the overall job processing time when applying both algorithms or the starvation-free algorithm alone is consistent for both the actual and augmented job logs. However, when only the preemptive task reallocation algorithm is applied, the wall time for the actual job log increased compared to when no algorithm was applied. This is because with fewer jobs, defragmentation can exacerbate blockages.

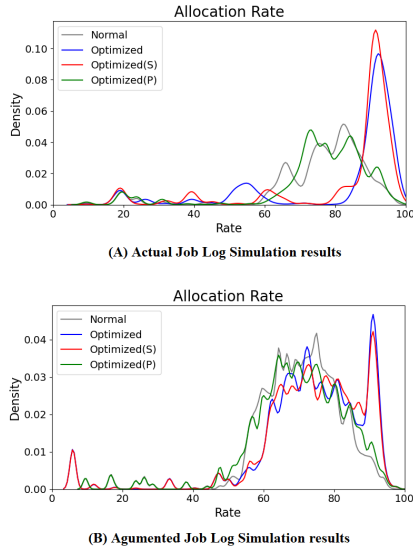


Fig. 10. Allocation rate comparison of algorithm combination.

However, from the perspective of the allocation rate, applying each algorithm individually to the actual job log achieves a higher allocation rate than applying no algorithm. This is shown in Fig. 10(A). Similarly, for the augmented job log, both individual and combined algorithms achieve higher allocation rates, as shown in Fig. 10(B). In Fig. 10(A), the

application of the preemptive task reallocation algorithm, denoted as Optimized(P), shows a slightly higher allocation rate, indicated by a slight shift to the right compared to the normal case. When only the starvation-free algorithm was applied, or both algorithms were applied, the allocation rate increased significantly, denoted as Optimized(S). In addition, when both algorithms are applied, a marginal shift to the right is observed, denoted as Optimized. The effectiveness of the algorithms is even more evident in the augmented job log simulation results shown in Fig. 10(B), where higher allocation rates are observed, even when only the starvation-free algorithm is applied.

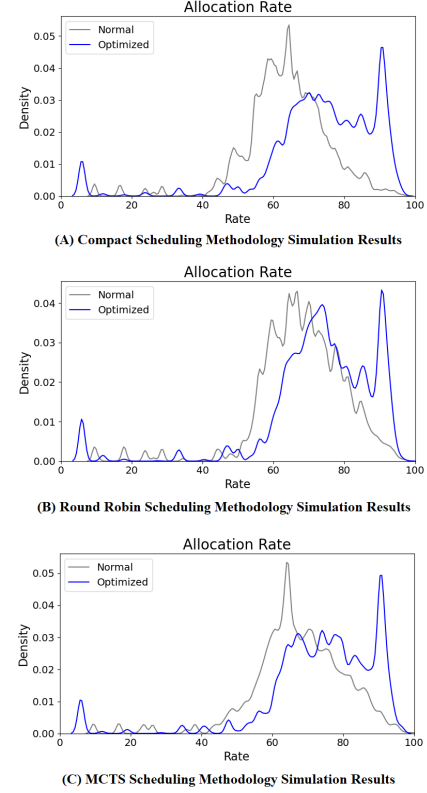


Fig. 11. Allocation rate comparison of various scheduling methodologies.

Queue adjustment and fragmentation curation handle the job-waiting queue and GPU server pool directly. Therefore, this is an agonistic mechanism against the scheduler methodologies. Previous evaluations used the most-allocated scheduler. Table V lists the queue adjustment and fragmentation curation archiving performance gains with the other three schedulers. An augmented job log was used; therefore, the same coefficients were used in the previous simulations that used an augmented job log.

Compact and round robin scheduling methodologies are simple. The MCTS-Monte Carlo Tree Search [12] is a search method based on randomness. Recently, several studies have introduced MCTS for resource scheduling, such as Hofmann1 C [13]. and Kung H [14]. In the simulations using MCTS in this study, the most-allocated method was employed. A

TABLE V
OTHER SCHEDULER METHODOLOGY APPLYING RESULTS

	Compact	Round robin	MCTS
Normal	1,700d 11h 01m	1,588d 18h 01m	1,605d 08h 01m
Optimal	1,462d 20h 30m	1,467d 08h 27m	1,462d 11h 31m
Gain ^a	238d (14%)	121d (7.6%)	143d (8.9%)

^aDays Count Only

comparison of the allocation rates for each scheduling methodology is shown in Fig. 11.

VII. CONCLUSIONS AND FUTURE WORKS

This study proposes a queue adjustment and fragmentation curation mechanism to maintain a high allocation rate in GPU server pools. Unlike conventional approaches used by general GPU schedulers, this mechanism reduces the occurrence of blockages that degrade scheduling efficiency and eliminates fragmentation in GPU server pools, thereby enhancing the efficiency of various schedulers. Comprising the starvation-free algorithm, which addresses blockages, and the preemptive task reallocation algorithm for defragmentation, queue adjustment, and fragmentation curation the wall time was reduced by 7.8% for the actual job log and 4.9% for the augmented job log.

Additionally, this study precisely defines the trade-offs introduced by these two algorithms and proposes methods for measuring them. To balance the trade-offs and performance gains, the coefficients α , β , ω , and δ were defined and calculated. The coefficients were derived not only for the job logs used in the experiments, but also for various probability distributions, demonstrating the versatility of the proposed mechanism. Queue adjustment and fragmentation curation mechanisms demonstrated wall time reductions of 4.9%, 14%, 7.6%, and 8.9% when applied alongside scheduling methods, such as most-allocated, compact, round robin, and MCTS, respectively. This effectiveness is attributed to the scheduler-neutral operation of the mechanism. The mechanism remains unaffected by the specific scheduling method used, which directly adjusts the job queue and server states.

However, identifying the optimal coefficients relies on a posterior approach involving numerous iterations, similar to hyperparameter tuning in deep learning. Although this method is suitable for research purposes, it presents a limitation in real-world scenarios, where coefficients must be precomputed to align with the workload characteristics. To address this limitation, future studies will focus on dynamically calculating the coefficients in real-time to determine and apply the most suitable coefficients in the current context. Given that these algorithms enable the precise calculation of costs and rewards, the application of reinforcement learning in real-time scenarios is also a promising direction for improvement.

REFERENCES

- [1] Y. Chen, Z. Guan, and X. Shao, "A comparative analysis of job scheduling algorithm," 2011 International Conference on Management Science and Industrial Engineering, MSIE 2011, pp. 1091–1095, 2011, doi: 10.1109/MSIE.2011.5707607.
- [2] D. S. Hochba, "Approximation Algorithms for NP-Hard Problems," ACM SIGACT News, vol. 28, no. 2, pp. 40–52, Jun. 1997, doi: 10.1145/261342.571216.
- [3] Z. Ye et al., "Deep Learning Workload Scheduling in GPU Datacenters: A Survey," ACM Comput Surv, vol. 56, no. 6, pp. 1–38, Jun. 2024, doi: 10.1145/3638757.
- [4] S. Jayaram Subramanya, D. Arfeen, S. Lin, A. Qiao, Z. Jia, and G. R. Ganger, "Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling," in Proceedings of the 29th Symposium on Operating Systems Principles, New York, NY, USA: ACM, Oct. 2023, pp. 642–657, doi: 10.1145/3600006.3613175.
- [5] Q. Hu, M. Zhang, P. Sun, Y. Wen, and T. Zhang, "Lucid: A Non-intrusive, Scalable and Interpretable Scheduler for Deep Learning Training Jobs," International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS, vol. 2, pp. 457–472, Jan. 2023, doi: 10.1145/3575693.3575705.
- [6] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," arXiv preprint arXiv:1909.08053, 2019.
- [7] L. Eyraud-Dubois and C. Bentes, "Algorithms for Preemptive Co-scheduling of Kernels on GPUs," Proceedings - 2020 IEEE 27th International Conference on High Performance Computing, Data, and Analytics, HiPC 2020, pp. 192–201, Dec. 2020, doi: 10.1109/HIPC50609.2020.00033.
- [8] "Linux Container." [Online]. Available: <http://linuxcontainers.org>
- [9] "Docker Hub." Accessed: Oct. 27, 2024. [Online]. Available: <https://hub.docker.com/>
- [10] "Hugging Face." Accessed: Oct. 27, 2024. [Online]. Available: <https://huggingface.co/>
- [11] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Curran Associates, Inc., 2017, pp. 4765–4774.
- [12] R. Coulom, "Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search," in Computers and Games, P. and D. H. H. L. M. (Jeroen) van den Herik H. Jaap and Ciancarini, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 72–83.
- [13] C. Hofmann, X. Liu, M. May, and G. Lanza, "Hybrid Monte Carlo tree search based multi-objective scheduling," Production Engineering, vol. 17, no. 1, pp. 133–144, Feb. 2023, doi: 10.1007/s11740-022-01152-9.
- [14] H.-L. Kung, S.-J. Yang, and K.-C. Huang, "An improved Monte Carlo Tree Search approach to workflow scheduling," Conn Sci, vol. 34, no. 1, pp. 1221–1251, Dec. 2022, doi: 10.1080/09540091.2022.2052265.